Chapter 18. Using Java with Other Languages

18.0 Introduction

Java has several methods of running programs written in other languages. You can invoke a compiled program or executable script using `Runtime.exec()`, as I'll describe in Recipe 18.1. There is an element of system dependency here, because you can only run external applications under the operating system they are compiled for. Alternatively, you can invoke one of a number of scripting languages (or *dynamic languages*)—running the gamut: AWK, bsh, Clojure, Ruby, Perl, Python, Scala—using javax.script, as illustrated in Recipe 18.3. Or you can drop down to C level with Java's *native code* mechanism and call between Java and compiled functions written in C/C++; see Recipes 18.6 and 18.7. Java 22 improves on this with the Foreign Function and Memory (FFM) interface, as discussed in Recipe 18.5. From native code, you can call out to functions written in just about any language. Not to mention that you can contact programs written in any language over a socket (see Chapter 15), with HTTP services (see Chapter 15), or with Java clients in RMI or CORBA clients in a variety of languages.

There is a wide range of other JVM languages, including these:

- BeanShell, a general scripting language for Java.
- Groovy, a Java-based scripting language that pioneered the use of closures in the Java language ecosystem. It also has a rapid-development web package called Grails and a build tool called Gradle (see Recipe 2.5).
- Jython, a full Java implementation of Python.
- JRuby, a full Java implementation of the Ruby language.
- Scala, a JVM language that claims to offer the "best of functional and OO" languages.
- Clojure, a predominantly functional Lisp-1 dialect for the JVM.
- Renjin (pronounced "R engine"), a fairly complete open source clone of the R statistics package with the ability to scale to the cloud. See Recipe 12.4 for an example using Renjin.

These are JVM-centric, and some can be called directly from Java to script, or vice versa, without using javax.script. A list of these languages can be found on Wikipedia.

Finally, a few languages provide their own mechanisms for interfacing with Java, such as Perl's `Inline::Java` module to call Java from Perl. These are not covered in *Java Cookbook*. As usual, to go the other way—calling Perl from Java—use javax.script, as in Recipe 18.3.

18.1 Running an External Program from Java

Problem

You want to run an external program from within a Java program.

Solution

Use one of the `exec()` methods in the `java.lang.Runtime` class. Or set up a `ProcessBuilder` and call its `start()` method.

Discussion

The `exec()` method in the `Runtime` class lets you run an external program. The command line you give is broken into strings by a simple `StringTokenizer` (see Recipe 3.1) and passed on to the operating system's "execute a program" system call. As an example, here is a simple program that uses `exec()` to run *kwrite*, a windowed text editor program.[1] On Windows, you'd have to change the name to `notepad` or `wordpad`, possibly including the full pathname, for example, *c:/windows/notepad.exe* (you can also use backslashes, but be careful to double those because the backslash is special in Java strings):

```
public class ExecDemoSimple {

 // Choose one of kate, kwrite, gedit, notepad, wordpad, ...
 static final String EDITOR = "kwrite";

 public static void main(String args[]) throws Exception {

  // Run a "notepad" style editor
  Process p = Runtime.getRuntime().exec(new String[]{EDITOR});

  p.waitFor();
 }
}
```

When you compile and run it, the appropriate editor window appears:

```
$ javac -d . ExecDemoSimple.java
$ java otherlang.ExecDemoSimple # causes a visual editor window to appear.
$
```

This version of `exec()` assumes that the pathname contains no blanks because these break proper operation of the `StringTokenizer`. To overcome this potential problem, use the overload of `exec()`, which takes an array of strings as arguments. Example 18-1 runs the Windows or Unix version of the Firefox web browser, assuming that Firefox was installed in the default directory (or another directory that is on your `PATH`). It passes the name of a help file as an argument, offering a kind of primitive help mechanism, as displayed in Figure 18-1.

Example 18-1. main/src/main/java/otherlang/ExecDemoBrowser.java

```
public class ExecDemoBrowser {
  private static final long serialVersionUID = 1;

  Logger logger = Logger.getLogger(ExecDemoBrowser.class.getName());

  /** The name of the browser. */
  private static final String BROWSER = "firefox";
  /** The name of the help file. */
  protected final static String HELP_FILE = "/help/index.html";
  /** The JFrame */
  protected final JFrame jf;

  /** main - instantiate and run */
  public static void main(String av[]) throws Exception {
    String program = av.length == 0 ? BROWSER : av[0];
    new ExecDemoBrowser(program);
  }

  /** The name of the binary executable that we will run */
  protected static String program;

  /** Constructor - set up strings and things. */
  public ExecDemoBrowser(String program) {
    jf = new JFrame("ExecDemoBrowser: " + program + " edition");
    ExecDemoBrowser.program = program;

    Container cp = jf.getContentPane();
    cp.setLayout(new FlowLayout());
    JButton b;
    cp.add(b=new JButton("Exec"));
    b.addActionListener(e -> runProgram());
    cp.add(b=new JButton("Wait"));
    b.addActionListener(e -> doWait());
    cp.add(b=new JButton("Exit"));
    b.addActionListener(e -> System.exit(0));
    jf.pack();
    jf.setLocation(400, 300);
    jf.setVisible(true);
  }

  Process process;
  ExecutorService threadPool = Executors.newFixedThreadPool(1);

  /** Start the help, in its own Thread. */
  public void runProgram() {

    threadPool.submit(() -> {
      try {
        // Get a "file:" URL for the Help File
        URL helpURL = ExecDemoBrowser.this.getClass().getResource(HELP_FILE);
        if (helpURL == null) {
          JOptionPane.showMessageDialog(jf,
```

```
            "Unable to find Help File in resource path",
            "Error",
            JOptionPane.ERROR_MESSAGE);
          System.exit(1);
        }

        // Start the external browser from the Java Application.

        String osName = System.getProperty("os.name");
        String run;
        if ("Mac OS X".equals(osName)) {
          run = "open -a " + program;
          // Any other OSes needing special handling?
        } else {
          run = program;
        }

        process = Runtime.getRuntime().exec(
          new String[]{run, helpURL.toString()});

        logger.info("In main after exec .");

      } catch (Exception ex) {
        JOptionPane.showMessageDialog(jf,
          "Error" + ex, "Error",
          JOptionPane.ERROR_MESSAGE);
        ex.printStackTrace();  // In terminal window, if any
      }
    });
  }

  public void doWait() {
    if (process == null) {
      logger.info("Nothing to wait for.");
      return;
    }
    logger.info("Waiting for process " + process);
    try {
      process.waitFor();
      // wait for process to complete
      // (may not work as expected for some old Windows programs)
      logger.info("Process " + process + " is done.");
    } catch (Exception ex) {
      JOptionPane.showMessageDialog(jf,
        "Error" + ex, "Error",
        JOptionPane.ERROR_MESSAGE);
    }
  }

}
```

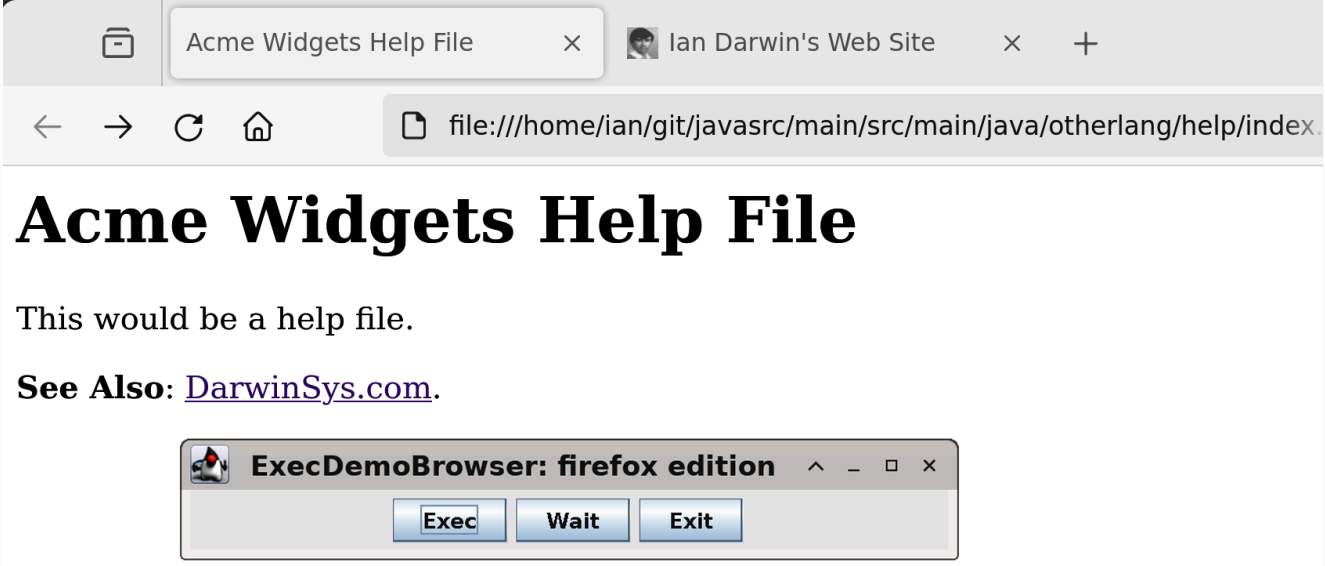


Figure 18-1. `ExecDemoBrowser` in action

Note that this particular example could be replaced by use of the `java.awt.Desktop` class. The Desktop API allows you to open a browser to a specified URL, launch the user's mail client (with an optional mailto URL), or launch the registered application to open/edit/print a file of a type that is known on the underlying operating system. Desktop is not discussed in this book; see the Javadoc or the example *desktop/src/main/java/desktop/DesktopDemo.java*.

A newer class, `ProcessBuilder`, replaces most nontrivial uses of `Runtime.exec()`. `ProcessBuilder` uses generic collections to let you modify or replace the environment, as shown in Example 18-2.

Example 18-2. main/src/main/java/otherlang/ProcessBuilderDemo.java

```
List<String> command = new ArrayList<>();          ❶
command.add("notepad");
command.add("foo.txt");
ProcessBuilder builder = new ProcessBuilder(command);❷
builder.environment().put("PATH",
      "/windows;/windows/system32;/winnt");         ❸
final Process godot = builder.directory(
   new File(System.getProperty("user.home"))).      ❹
   start();
System.err.println("Waiting for Godot");            ❺
godot.waitFor();                                    ❻
```

❶ Set up the command-line argument list: editor program name and filename.

❷ Use that to start configuring the `ProcessBuilder`.

❸ Configure the builder's environment to a list of common Windows directories.

❹ Set the initial directory to the user's home, and start the process!

❺ I always wanted to be able to use this line in code.

❻ Wait for the end of our little play.

For more on `ProcessBuilder`, see the Javadoc for `java.lang.ProcessBuilder`.

18.2 Running a Program and Capturing Its Output

Problem

You want to run a program but also capture its output.

Solution

Use the `Process` object's `getInputStream()`; read and copy the contents to `System.out` or wherever you want them.

Discussion

The original concept of standard output and standard error was that they would always be connected to the terminal; this dates from an earlier time when almost all computer users worked at the command line. Today, a program's standard output and error output do not always automatically appear anywhere. Arguably, there should be an automatic way to make this happen. But for now, you need to add a few lines of code to grab the program's output and print it:

```
public class ExecDemoLs {

  private static Logger logger =
    Logger.getLogger(ExecDemoLs.class.getSimpleName());

  /** The program to run */
  public static final String PROGRAM = "ls"; // "dir" for Windows
  /** Set to true to end the loop */
  static volatile boolean done = false;

  public static void main(String argv[]) throws IOException {

    final Process p;     // Process tracks one external native process
    BufferedReader is;  // reader for output of process
    String line;

    p = Runtime.getRuntime().exec(new String[]{PROGRAM});

    logger.info("In Main after exec");

    try {
      p.waitFor();
    } catch (InterruptedException ex) {
      // Unlikely, but OK, just quit.
      return;
    }

    // getInputStream gives an Input stream connected to
```

```
    // the process p's standard output (and vice versa). We use
    // that to construct a BufferedReader so we can readLine() it.

    is = new BufferedReader(new InputStreamReader(p.getInputStream()));

    while (!done && ((line = is.readLine()) != null))
      System.out.println(line);

    logger.info("In Main after EOF");

    return;
  }
}
```

This is such a common occurrence that I've packaged it up into a class called `ExecAndPrint`, which is part of my com.darwinsys.lang package. `ExecAndPrint` has several overloaded forms of its `run()` method (see the documentation for details), but they all take at least a command and optionally an output file to which the command's output is written. Example 18-3 shows the code for some of these methods.

Example 18-3. darwinsys-api/src/main/java/com/darwinsys/lang/ExecAndPrint.java

```
/** Need a Runtime object for any of these methods */
protected final static Runtime r = Runtime.getRuntime();

/** Run the command given as a String, output to System.out
 * @param cmd The command
 * @return The command's exit status
 * @throws IOException if the command isn't found
 */
public static int run(String cmd) throws IOException {
  return run(cmd, new OutputStreamWriter(System.out));
}

/** Run the command given as a String, output to "out"
 * @param cmd The command and list of arguments
 * @param out The output file
 * @return The command's exit status
 * @throws IOException if the command isn't found
 */
public static int run(String cmd, Writer out) throws IOException {

  Process p = r.exec(cmd);

  FileIO.copyFile(new InputStreamReader(p.getInputStream()), out, true);
  try {
    p.waitFor();  // wait for process to complete
  } catch (InterruptedException e) {
    return -1;
  }
  return p.exitValue();
}
```

As a simple example of using `exec()` directly along with `ExecAndPrint`, I'll create three temporary files, list them (directory listing), and then delete them. The program to do this is shown in Example 18-4.

Example 18-4. main/src/main/java/otherlang/ExecDemoFiles.java

```
// Get and save the Runtime object.
Runtime rt = Runtime.getRuntime();

// Create three temporary files (the slow way!)
rt.exec(new String[]{"mktemp", "file1"});
rt.exec(new String[]{"mktemp", "file2"});
rt.exec(new String[]{"mktemp", "file3"});

// Run the "ls" (directory lister) program
// with its output sent into a file
String[] args = { "ls", "-l", "file1", "file2", "file3" };
ExecAndPrint.run(args);

rt.exec(new String[]{"rm", "file1", "file2", "file3"});
```

When I run the `ExecDemoFiles` program, it lists the three files it has created:

```
-rw-------  1 ian  wheel  0 Jan 29 14:29 file1
-rw-------  1 ian  wheel  0 Jan 29 14:29 file2
-rw-------  1 ian  wheel  0 Jan 29 14:29 file3
```

A process isn't necessarily destroyed when the Java program that created it—the "parent process"—exits or bombs out. Simple text-based programs will be, but window-based programs like a graphical text editor, browser, or a Java-based `JFrame` or JavaFX application, will not. For example, our `ExecDemoBrowser` program started a web browser, and when `ExecDemoBrowser`'s Exit button is clicked, `ExecDemoBrowser` exits but the browser stays running. What if you want to be sure a process has completed? The `Process` object has a `waitFor()` method that lets you do so, and an `exitValue()` method that tells you the return code from the process. Finally, should you wish to forcibly terminate the other process, you can do so with the `Process` object's `destroy()` method, which takes no argument and returns no value. Example 18-5 is `ExecDemoWait`, a program that runs whatever program you name on the command line (along with arguments), captures the program's standard output, and waits for the program to terminate.

Example 18-5. main/src/main/java/otherlang/ExecDemoWait.java

```
// A Runtime object has methods for dealing with the OS
Runtime r = Runtime.getRuntime();
Process p;     // Process tracks one external native process
BufferedReader is;  // reader for output of process
String line;

// Our argv[0] contains the program to run; remaining elements
// of argv contain args for the target program. This is just
// what is needed for the String[] form of exec.
```

```
        p = r.exec(argv);

        System.out.println("In Main after exec");

        // getInputStream gives an Input stream connected to
        // the process p's standard output. Just use it to make
        // a BufferedReader to readLine() what the program writes out.
        is = new BufferedReader(new InputStreamReader(p.getInputStream()));

        while ((line = is.readLine()) != null)
          System.out.println(line);

        System.out.println("In Main after EOF");
        System.out.flush();
        try {
          p.waitFor();  // wait for process to complete
        } catch (InterruptedException e) {
          System.err.println(e);  // "Can't Happen"
          return;
        }
        System.err.println("Process done, exit status was " + p.exitValue());
```

See Also

You wouldn't normally use any form of `exec()` to run one Java program from another in this way; instead, you'd probably create it as a thread within the same process, because this would be faster (the Java interpreter is already up and running, so why wait for another copy of it to start up?). See Chapter 11.

When building industrial-strength applications, note the cautionary remarks in the Java API docs for the `Process` class concerning the danger of losing some of the I/O due to insufficient buffering by the operating system.

18.3 Calling Other Languages via javax.script

Problem

You want to invoke a script written in some other language from within your Java program, running in the JVM, with the ability to pass variables directly to/from the other language.

Solution

If the script you want is written in any of the two-dozen-plus supported languages, use javax.script. Those languages include AWK, Perl, Python, Ruby, BeanShell, Pnuts, Ksh/Bash, R (Renjin), and several implementations of JavaScript.

Discussion

The Scripting Engine mechanism was once quite popular, but it's less so now. One of the first tasks when using this API is to find out the installed scripting engines, and then pick one that is available. The `ScriptEnginesList` program in Example 18-6 lists the engines on the `CLASSPATH` in this project.

Example 18-6. main/src/main/java/otherlang/ScriptEnginesList.java

```
        ScriptEngineManager scriptEngineManager = new ScriptEngineManager();
        System.out.println("Available script engines are: ");
        scriptEngineManager.getEngineFactories().forEach(factory ->
            System.out.println(factory.getLanguageName()));
```

The output was:

```
Available script engines are:
R
SimpleCalc
python
```

As shown, my system at the time of this run had engines named `R`, `SimpleCalc`, and `python`. Conspicuously absent in the output is an engine for ECMAScript (a.k.a. JavaScript). For decades Java would always provide this engine, but no more. Thus the program in Example 18-7 requires the Nashorn JAR file on `CLASSPATH`, which is generously provided by the *pom.xml* file for this project.

Example 18-7. main/src/main/java/otherlang/ScriptEnginesDemo.java

```
public class ScriptEnginesDemo {

  public static void main(String[] args) throws ScriptException {
    ScriptEngineManager scriptEngineManager = new ScriptEngineManager();

    // Run a script in the JavaScript language
    String lang = "JavaScript";
    ScriptEngine engine =
      scriptEngineManager.getEngineByName(lang);
    if (engine == null) {
      System.err.println("Could not find engine");
      return;
    }
    engine.eval("print(\"Hello from " + lang + "\");");
  }
}
```

Example 18-8 is a simple demo of calling Python from Java using `javax.scripting`. We know the name of the scripting engine we want to use: Python. We'll use the JVM implementation known as Jython, which was originally called JPython but was changed due to a trademark issue. Once we add the dependency

`org.python:jython-slim:2.7.3` (or later) into the build file, the script engine will be detected when running the program.

Example 18-8. main/src/main/java/otherlang/PythonFromJava.java

```
/**
 * Demo using Python (Jython) to get a Java variable, print, and change it.
 * @author Ian Darwin
 */
public class PythonFromJava {
 private static final String PY_SCRIPTNAME = "pythonfromjava.py";

 public static void main(String[] args) throws Exception {
  ScriptEngineManager scriptEngineManager = new ScriptEngineManager();

  ScriptEngine engine = scriptEngineManager.getEngineByName("python");
  if (engine == null) {
        System.out.println("" +
      "Could not find 'python' engine; add its JAR to CLASSPATH");
   System.exit(1);
  }

  final Bindings bindings = engine.getBindings(ScriptContext.ENGINE_SCOPE);
  bindings.put("meaning", 42);

  // Let's run a Python script stored on disk (well, on classpath):
  InputStream is =
   PythonFromJava.class.getResourceAsStream("/" + PY_SCRIPTNAME);
  if (is == null) {
   throw new IOException("Could not find file " + PY_SCRIPTNAME);
  }
  engine.eval(new InputStreamReader(is));
  System.out.println("Java: Meaning of life is now " + bindings.get("meaning"));
 }
}
```

Here is the Python script file, from *src/main/resources/pythonfromjava.py*:

```
# Simple Hello in Python
print('Hello from Python')
global meaning
print('Python: Meaning of Life is ' + str(meaning))
# Python doesn't do strong typing:
meaning = 'Good morning'
```

Running the Java program under an IDE yields the following output:

```
Hello from Python
Python: Meaning of Life is 42
Java: Meaning of life is now Good morning
```

The second engine, `R`, implements the R language as described in Recipe 12.4.

The third engine, the trivial and imaginatively named `SimpleCalc`, is just to show that it is not terribly difficult to build a scripting engine. See my write-up on my website. The file *main/src/main/java/otherlang/SimpleCalcDemo.java* demonstrates my basic `ScriptEngine`, whose source can be found in the *calcscriptengine* subdirectory.

See Also

Before Oracle dismantled *java.net*, there was a list of script engines for many languages (see the archived list; the links don't work, but it shows the extent of the languages that were available). Back then, you could download the script engines from that site. I am not aware of a current official list of engines, nor a download site, unfortunately. However, the list maintained as part of the scripting project can be found in an unofficial source code repository, which I have a copy of in my GitHub repository. It should be possible to build the engine you want from that repository (look in the *engines* subdirectory). A dozen or so other engines are maintained by others outside this project; for example, there is a `Perl5` script engine from Google Code, and an engine that supports compilation of Java(!), available on GitHub.

There is a also a list of Java-compatible scripting languages (not necessarily all using javax.script).

18.4 Mixing Languages with GraalVM 21

Problem

GraalVM aims to be multilanguage, and you'd like to use different languages in the VM.

Solution

Install the current Graal VM as described in Recipe 1.12. Download and use the additional language(s) via Maven or Gradle.

Discussion

While GraalVM positions itself as able to support a wide variety of programming languages, the number currently supported is small but growing. Let's try invoking Python code from within Java. Assuming you've installed Graal itself as per Recipe 1.12, you can add additional languages via Maven or Gradle:

```
<dependencies>
  <dependency>
    <groupId>org.graalvm.polyglot</groupId>
    <artifactId>polyglot</artifactId>
    <version>${graalvm.version}</version>
  </dependency>
  <dependency>
    <groupId>org.graalvm.polyglot</groupId>
    <artifactId>js</artifactId>
```

```
      <version>${graalvm.version}</version>
      <type>pom</type>
    </dependency>
    <dependency>
      <groupId>org.graalvm.polyglot</groupId>
      <artifactId>python</artifactId>
      <version>${graalvm.version}</version>
      <type>pom</type>
    </dependency>
    <dependency>
      <groupId>org.graalvm.polyglot</groupId>
      <artifactId>tools</artifactId>
      <version>${graalvm.version}</version>
      <type>pom</type>
    </dependency>
  </dependencies>
```

Then the code in Example 18-9 can be built and run, either under the JDK or under Maven. With Maven, the program may appear not to terminate; you can wait about 10 seconds, or use Ctrl+C to interrupt it.

Example 18-9. graal/src/main/java/graaldemo/JavaCallPython.java

```
import org.graalvm.polyglot.Context;
import org.graalvm.polyglot.Value;

/**
 * GraalVM polyglot: calling Python from Java.
 */
public class JavaCallPython {

  public static void main(String[] args)  {

    try (Context context = Context.create("python")) {
      Value result = context.eval("python", "2 + 2");
      int i = result.asInt();
      System.out.println(i);
    }
  }
}
```

This has been run in an IDE and with:

```
mvn exec:java -Dexec.mainClass=graaldemo.JavaCallPython
```

Unsurprisingly, it prints `4`. The online source includes a similar program that calls JavaScript from Java. A more complete "starter kit" that also calls Ruby and other languages, and uses a Maven plug-in to build the native image, can be found at the GraalVM GitHub repository.

TIP

Graal versions prior to 21 used a command-line tool called `gu` (graal updater) that installed the language packs directly into the JVM, which created maintenance problems. So `gu` was discontinued and replaced by the language packs in Maven Central.

18.5 Calling Between Java and Native Code with the Foreign Function and Memory API 22

Problem

You want easier access to non-Java compiled code.

Solution

Use the Foreign Function and Memory API (FFI/FFM) to connect to functionality provided by compiled libraries in other languages.

Discussion

The Foreign Function and Memory API is described in JEP 454 as a means to access code outside of Java's purview. This was created as part of Project Panama and is often abbreviated to FFI because that's what it's called in many other popular languages, but Java documentation officially refers to it as FFM. The API lets you pass values into C code and retrieve values from the C code. The latter must be compiled, using a C compiler compatible with the one used to build the JDK, and formed into a shared library (`.so` , `.dll` , `.dylib` , depending on the operating system). In addition, you must pay attention to the sizes of values like `int` and `long` , which in C have different sizes on different CPU families.

Example 18-10 is the C code used in this demo. Obviously created by some kind of Monty Python fan, this provides a (very small) subset of the troupe's comedy skits or sketches, and two functions, one for replacing a given sketch title by index number, and another for retrieving the sketch whose title is currently in the given slot. If you don't know C, don't fret too much. It's where a lot of Java's syntax originated, so it shouldn't be completely opaque, and you can see the two described functions (as well as a `main` function to demonstrate the methods' basic functioning).

Example 18-10. main/src/main/java/ffi/hello-ffi.c

```
/*
 * A silly library just for the FFI demo.
 */

#include <stdio.h>
#include <string.h>

char* sketches[] = {
  "Dead Parrot Sketch",
```

```
  "Ministry of Silly Walks",
  "The Funniest Joke In The World",
  "Hell's Grannies"
};

int nSketches = sizeof(sketches)/sizeof(char*);

int updatesketch(int n, char* newVal) {
  if (n > nSketches) {
    fprintf(stderr, "Try to update non-existent sketch!");
    return -1;
  }
  sketches[n] = strdup(newVal);
  printf("Setting %d to %s\n", n, newVal);
  return n;
}

char* pythonsketch(int n) {
  if (n > nSketches)
    return "Very silly";
  return sketches[n];
}

/* A main program, just to show that the function works */
int main(int argc, char **argv) {
  updatesketch(2, "Always look on the bright side of life");
  puts(pythonsketch(2));
}
```

The Makefile is pretty basic, but the arguments passed to the C compiler may need to be changed on other operating systems. The Makefile does this for our Java-calling-C example on my system:

```
cc -shared -fPIC hello-ffi.c -o hello-ffi.so
```

The Java code has to deal with several concepts and several classes, most in `java.lang.foreign`:

- *Down call* refers to a call from Java down to C level; *up call* refers to a call from native code up to Java (typically a callback).
- Heap memory is allocated by Java's `new` operator—every time you create an object in Java, its storage is in the Java-managed heap. This memory is eligible for garbage collection when no longer in use.
- Off-heap memory is actually allocated in the OS-level heap area (so the term is a bit of a misnomer), typically by system-level functions like `malloc()` in the standard C library. Off-heap memory is never garbage collected by Java. It is thus possible to have memory leaks.
- An `Arena` is responsible for allocating and managing a chunk of off-heap memory; some versions can arrange to be freed up.
- A `MemorySegment` represents a contiguous segment of memory within an `Arena`.
- A `Linker` lets you load shared objects and find functions within them.
- A `FunctionDescriptor` represents the signature of one function in a C library.

- A `MethodHandle` (from `java.lang.invoke`, covered in Recipe 17.5) is a reference to an executable code region (e.g, function).

There are several types of `Arena`s:

- The most common is *confined* (obtained with `Arena.ofConfined()`). These must be closed (commonly via `try-with-resources`) when done, and they are only accessible to the thread that created them.
- A *shared* arena, created with `Arena.ofShared()`, is, as the name implies, shareable across threads, any of which may close it.
- An *automatic* arena, created with `Arena.ofAuto()`, is shareable, and as the name implies, will be deallocated automatically when eligible for garbage collection. It cannot be closed explicitly.
- A *global* arena, created with `Arena.global()`, is shareable and never deallocated. It cannot be closed at all.

With this background, Example 18-11 is a Java program that calls both functions in the C code and displays the results.

Example 18-11. main/src/main/java/ffi/CallCFromJava.java

```
/**
 * Demo program calling two methods in C code, updatesketch() and
 * pythonsketch(). The C code hello-ffi.c has a list of strings that pythonsketch()
 * can retrieve by index number, and an updatesketch() that allows putting
 * a new sketch title into the slot by number. So we update slot #2
 * and then retrieve it to prove that we successfully passed the string
 * in, had it stored by the C code, and that we then retrieved it.
 */
public class CallCFromJava {
 void main() throws Throwable {
  String newSketch = "Killer Bunny";

  try (Arena arena = Arena.ofConfined()) {
   // Allocate off-heap memory and copy 'message' into it
   MemorySegment nativeString = arena.allocateUtf8String(newSketch);
   // Name of C function to find and invoke
   String updateMethod = "updatesketch", getMethod = "pythonsketch";
   // First, get an instance of the native linker
   Linker linker = Linker.nativeLinker();
   // Now get the address of the C function signature
   SymbolLookup ourLib = SymbolLookup.libraryLookup(
    "/home/ian/git/javasrc/main/src/main/java/ffi/hello-ffi.so", arena);
   Optional<MemorySegment> segment = ourLib.find(updateMethod);
   if (segment.isEmpty()) {
    throw new IllegalArgumentException(
     "Method " + updateMethod + " not found!");
   }
   MemorySegment foreignFuncAddr = segment.get();
   // Create argument-list description of the C function
   FunctionDescriptor update_sig=
     FunctionDescriptor.of(ValueLayout.JAVA_LONG, // return
       ValueLayout.JAVA_LONG,  // first argument
       ValueLayout.ADDRESS);   // second
```

```
    // Obtain a downcall handle for the C function
    MethodHandle strlen =
     linker.downcallHandle(foreignFuncAddr, update_sig);
    // Finally, what you've all been waiting for:
    //  Call a C function directly from Java
    var ret = (long) strlen.invokeExact(2L, nativeString);
    System.out.println("Update function returned " + ret);

    // Now call getSketch(2) to prove that it got updated
    segment = ourLib.find(getMethod);
    if (segment.isEmpty()) {
     throw new IllegalArgumentException(
       "Method " + getMethod + " not found!");
    }
    foreignFuncAddr = segment.get();
    // Create argument-list description of the C function
    update_sig=
       FunctionDescriptor.of(ValueLayout.ADDRESS, // return
          ValueLayout.JAVA_LONG);  // only input argument
    // Obtain a downcall handle for the C function
    MethodHandle getter = linker.downcallHandle(foreignFuncAddr,
       update_sig);
    // Once again we call a C function directly from Java
    var retStr = (MemorySegment)getter.invokeExact(2L);
    Consumer<MemorySegment> cleanup = _ -> {
     // Maybe some cleanup here what we had allocated in native space
    };
    retStr = retStr.reinterpret(newSketch.length() + 1, arena, cleanup);
    System.out.println("Update function returned " + retStr);
    String updatedValue = retStr.getUtf8String(0);
    System.out.println("Retrieved updatedValue as " + updatedValue);
   }
  }
}
```

There is quite a bit more to the FFM API, including support for `struct` and `union` types. Refer to the documentation for the java.lang.foreign package and/or to the *FFM Guide* at Oracle, which is Section 12 of the *Java Platform Core Libraries*.

18.6 Calling Other Languages via Native Code (JNI)

Problem

You wish to call native C/C++ functions from Java, either for efficiency or to access hardware- or system-specific features. And you can't use the Java 22+ Foreign Function and Memory API described in Recipe 18.5.

Solution

Use the Java Native Interface (JNI). But note that JNI was superceded in Java 22+ with the Foreign Function and Memory interface described in Recipe 18.5.

Discussion

Java lets you load native or compiled code into your Java program. Why would you want to do such a thing? The best reason would probably be to access OS-dependent functionality or existing code written in another language. A not-as-good reason would be speed: native code can sometimes run faster than Java, though this is becoming less important as computers get faster and more multicore and as the dynamic optimizations of the JVM become more sophisticated.

The native code language bindings are defined for code written in C or C++. If you need to access a language other than C/C++, write a bit of C/C++ and have it pass control to other functions or applications, using any mechanism defined by your operating system.

Due to such system-dependent features as the interpretation of header files and the allocation of the processor's general-purpose registers, your native code may need to be compiled by the same C compiler used to compile the Java runtime for your platform. For example, on Solaris you can use SunPro C or maybe gcc. On Win32 platforms, use Microsoft Visual C++ version 4.x or higher (32 bit). For Linux and macOS, you should be able to use the provided gcc-based compiler. For other platforms, see your Java vendor's documentation.

Also note that the details in this section are for the JNI of Java 1.1 and later, which differs in some details from 1.0 and from Microsoft's native interface.

IAN'S BASIC STEPS: JAVA CALLING NATIVE CODE

To call native code from Java, follow these steps:

1. Write Java code that calls a native method.
2. Compile this Java code.
3. Create an *.h* file using *javah*.
4. Write a C function that does the work.
5. Compile the C code into a loadable object.
6. Try it!

Step 1 is to write Java code that calls a native method. To do this, use the keyword `native` to indicate that the method is native, and provide a static code block that loads your native method using `System.loadLibrary()`. (The dynamically loadable module is created in step 5.) Static blocks are executed when the class containing them is loaded; loading the native code here ensures it is in memory when needed.

Object variables that your native code can modify should carry the `volatile` modifier. The file *HelloJni.java*, shown in Example 18-12, is a good starting point.

Example 18-12. main/src/main/java/jni/HelloJni.java

```
/**
 * A trivial class to show Java Native Interface 1.1 usage from Java.
 */
public class HelloJni {
 int myNumber = 42; // used to show argument passing

 // declare native method
 public native void displayHelloJni();

 // Application main, call its display method
 public static void main(String[] args) {
  System.out.println("HelloJni starting; args.length="+
                args.length+"...");
  for (int i=0; i<args.length; i++)
                System.out.println("args["+i+"]="+args[i]);
  HelloJni hw = new HelloJni();
  hw.displayHelloJni();// call the native function
  System.out.println("Back in Java, \"myNumber\" now " + hw.myNumber);
 }

 // Static code blocks are executed once, when class file is loaded
 static {
  System.loadLibrary("hello");
 }
}
```

Step 2 is simple; just use `javac HelloJni.java` as you normally would. You probably won't get any compilation errors on a basic program like this; if you do, correct them and try the compilation again.

Step 3 requires you to create an *.h* file. On modern versions of Java, this functionality is included in `javac`. The current functionality is used as:

```
$ javac -h output_dir SomeFile.java
```

That is, the `-h` option requires a directory argument (which can be "`.`" but must be specified). This both compiles the Java source into a class file as usual and produces the *.h* file for writing the C code.

In very old versions of Java (up to Java 8), you'd use `javah` to produce this file:

```
$ javah jni.HelloJni        // produces HelloJni.h
```

The *.h* file produced is a glue file, not really meant for human consumption and particularly not for editing. But by inspecting the resulting *.h* file, you'll see that the C method's name is composed of the name `Java`, the package name (if any), the class name, and the method name:

```
JNIEXPORT void JNICALL Java_HelloJni_displayHelloWorld(JNIEnv *env,
  jobject this);
```

Step 4 has you create a C function that does the work. You must use the same function signature as is used in the *.h* file. This function can do whatever it wants. Note that it is passed two arguments: a JVM environment variable and a handle for the `this` object. Table 18-1 shows the correspondence between Java types and the C types (JNI types) used in the C code.

Table 18-1. Java and JNI types

Java type	JNI	Java array type	JNI
byte	jbyte	byte[]	jbyteArray
short	jshort	short[]	jshortArray
int	jint	int[]	jintArray
long	jlong	long[]	jlongArray
float	jfloat	float[]	jfloatArray
double	jdouble	double[]	jdoubleArray
char	jchar	char[]	jcharArray
boolean	jboolean	boolean[]	jbooleanArray
void	jvoid		
Object	jobject	Object[]	jobjectArray
Class	jclass		
String	jstring		
array	jarray		
Throwable	jthrowable		

Example 18-13 is a complete C native implementation. Passed an object of type `HelloJni`, it increments the integer `myNumber` contained in the object.

Example 18-13. main/src/main/java/jni/HelloJni.c

```
#include <jni.h>
#include "jni_HelloJni.h"
#include <stdio.h>
#include <stdlib.h>
/*
 * This is the Java Native implementation of displayHelloJni.
 */
JNIEXPORT void JNICALL Java_jni_HelloJni_displayHelloJni(JNIEnv *env, jobject this) {
 jfieldID fldid;
 jint n, nn;
```

```
  (void)printf("Hello from a Native Method\n");

  if (this == NULL) {
    fprintf(stderr, "'this.' pointer is null!\n");
    return;
  }
  if ((fldid = (*env)->GetFieldID(env,
       (*env)->GetObjectClass(env, this), "myNumber", "I")) == NULL) {
    fprintf(stderr, "GetFieldID failed");
    return;
  }

  n = (*env)->GetIntField(env, this, fldid);/* retrieve myNumber */
  printf("\"myNumber\" value is %d\n", n);

  (*env)->SetIntField(env, this, fldid, ++n);/* increment it! */
  nn = (*env)->GetIntField(env, this, fldid);

  printf("\"myNumber\" value now %d\n", nn); /* make sure */
  return;
}
```

Step 5 is to compile the C code into a loadable object. Naturally, the details depend on platform, compiler, etc. For example, on Windows, you could use something like this:

```
> set JAVA_HOME=C:\java               # or wherever
> set INCLUDE=%JAVA_HOME%\include;%JAVA_HOME%\include\Win32;%INCLUDE%
> set LIB=%JAVA_HOME%\lib;%LIB%
> cl HelloJni.c -Fehello.dll -MD -LD
```

And on Unix, you would use something like this:

```
$ export JAVAHOME=/local/java    # or wherever
$ cc -I$JAVAHOME/include -I$JAVAHOME/include/solaris \
      -G HelloJni.c -o libhello.so
```

Example 18-14 is a Unix Makefile that puts all the build steps in one place. You just have to type `make` to build all the pieces, and they are ready to run. The definition of `INCLUDES` needs to be tailored for different locations of the C-language header files.

Example 18-14. main/src/main/java/jni/Makefile (Unix version)

```
# Configuration Section

#CFLAGS_FOR_SO = -G # Solaris
CFLAGS_FOR_SO = -shared
CSRCS    = HelloJni.c
# JAVA_HOME should have been set in the environment
```

```
#INCLUDES  = -I$(JAVA_HOME)/include -I$(JAVA_HOME)/include/solaris
#INCLUDES  = -I$(JAVA_HOME)/include -I$(JAVA_HOME)/include/openbsd
INCLUDES  = -I$(JAVA_HOME)/include -I/Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/include/darwin/
LIBDIR=$(JAVA_HOME)/lib/server

all:   testhello testjavafromc

# This part of the Makefile is for C called from Java, in HelloJni
testhello:   hello.all
   @echo
   @echo "Here we test the Java code \"HelloJni\" that calls C code."
   @echo
   LD_LIBRARY_PATH=$(LIBDIR):. java HelloJni.java

hello.all:   HelloJni.class libhello.so

HelloJni.class: HelloJni.java
   javac -h . HelloJni.java

HelloJni.o:: jni_HelloJni.h

libhello.so: $(CSRCS) jni_HelloJni.h
 $(CC) -L$(LIBDIR) $(INCLUDES) $(CFLAGS_FOR_SO) $(CSRCS) -o libhello.so

# This part of the Makefile is for Java called from C, in javafromc
testjavafromc: javafromc.all hello.all
 @echo
 @echo "Now we test HelloJni using javafromc instead of java"
 @echo
 LD_LIBRARY_PATH=$(LIBDIR):. ./javafromc HelloJni
 @echo
 @echo "That was, in case you didn't notice, C->Java->C. And,"
 @echo "incidentally, a replacement for JDK program \"java\" itself!"
 @echo "(not a complete one, nor a very good one, but OK for simple cases)"
 @echo


javafromc.all: javafromc

javafromc: javafromc.o
 $(CC) -L$(LIBDIR) -L . javafromc.o -ljvm -o $@

javafromc.o: javafromc.c
 $(CC) -c $(INCLUDES) javafromc.c

clean:
 rm -f core *.class *.o *.so jni_HelloJni.h
clobber: clean
 rm -f javafromc
```

Step 6: you're done! Just run the Java interpreter on the class file containing the main program. Assuming that you've set whatever system-dependent settings are necessary (possibly including both `CLASSPATH` and `LD_LIBRARY_PATH` or its equivalent), the program should run as follows:

```
C&gt; java jni.HelloJni
Hello from a Native Method      // from C
"myNumber" value is 42          // from C
"myNumber" value now 43         // from C
Value of myNumber now 43        // from Java
C&gt;
```

Congratulations! You've called a native method. However, you've given up portability; the Java class file now requires you to build a loadable object for each operating system and hardware platform. Multiply {Windows, macOS, Sun Solaris, HP/UX, Linux, OpenBSD, NetBSD, FreeBSD} times {Intel-32, Intel-64/AMD64, ARM, ARM64, and maybe SPARC64, PowerPC, and HP-PA}, and you begin to see the portability issues.

Beware that problems with your native code can and will crash the runtime process right out from underneath the Java Virtual Machine. The JVM can do nothing to protect itself from poorly written C/C++ code. Memory must be managed by the programmer; there is no automatic garbage collection of memory obtained by the system runtime allocator. You're dealing directly with the operating system and sometimes even the hardware, so be careful. Be very careful.

See Also

If you need more information on Java native methods, you might be interested in the comprehensive treatment found in *Essential JNI: Java Native Interface* by Rob Gordon (Prentice Hall).

18.7 Calling Java from Native Code with JNI

Problem

You need to go the other way, calling Java from C/C++ code.

Solution

Use the Java Native Interface (JNI) again.

Discussion

JNI provides an interface for calling Java from C, with calls to:

1. Create a JVM
2. Load a class
3. Find and call a method from that class (e.g., `main`)

JNI lets you add Java to legacy code. That can be useful for a variety of purposes and lets you treat Java code as an extension language.

The code in Example 18-15 takes a class name from the command line, starts up the JVM, and calls the `main()` method in the class.

Example 18-15. main/src/main/java/jni/javafromc.c (calling Java from C)

```
#include <stdio.h>
#include <stdlib.h>
#include <jni.h>

/*
 * This is a C program that calls Java code.
 * This could be used as a model for building Java into an
 * existing application as an extention language, for example.
 */
int main(int argc, char *argv[]) {
 int i;
 JavaVM *jvm;    /* The Java VM we will use */
 JNIEnv *myEnv;    /* pointer to native environment */
 JavaVMInitArgs jvmArgs; /* JNI initialization arguments */
 jclass myClass, stringClass;  /* pointer to the class type */
 jmethodID myMethod;  /* pointer to the main() method */
 jarray args;    /* becomes an array of Strings */
 jthrowable tossed;  /* Exception object, if we get one. */

 /* initialize the JVM! */
 JavaVMOption options[2];
 options[0].optionString = "-Xmx512m";
 options[1].optionString = "-XX:NonNMethodCodeHeapSize=128m";
 jvmArgs.version = JNI_VERSION_1_6;
 jvmArgs.nOptions = 1;
 jvmArgs.options = options;
 jvmArgs.ignoreUnrecognized = 1;

 if (JNI_CreateJavaVM(&jvm, (void **)&myEnv, &jvmArgs) < 0) {
  fprintf(stderr, "CreateJVM failed\n");
  exit(1);
 }

 /* find the class named in argv[1] */
 if ((myClass = (*myEnv)->FindClass(myEnv, argv[1])) == NULL) {
  fprintf(stderr, "FindClass %s failed\n", argv[1]);
  exit(1);
 }

 /* find the static void main(String[]) method of that class */
 myMethod = (*myEnv)->GetStaticMethodID(
  myEnv, myClass, "main", "([Ljava/lang/String;)V");
 if (myMethod == NULL) {
  fprintf(stderr, "GetStaticMethodID failed\n");
  exit(1);
 }

 /* Since we're calling main, must pass along the command-line arguments,
  * in the form of Java String array */
```

```
  if ((stringClass = (*myEnv)->FindClass(myEnv, "java/lang/String")) == NULL){
    fprintf(stderr, "get of String class failed!!\n");
    exit(1);
  }

  /* make an array of Strings, - 1 for progname & 1 for the class name */
  if ((args = (*myEnv)->
      NewObjectArray(myEnv, argc-2, stringClass, NULL))==NULL) {
    fprintf(stderr, "Create array failed!\n");
    exit(1);
  }

  /* fill the array */
  for (i=2; i<argc; i++)
    (*myEnv)->SetObjectArrayElement(myEnv,
      args, i-2, (*myEnv)->NewStringUTF(myEnv, argv[i]));

  /* finally, call the method. */
  (*myEnv)->CallStaticVoidMethodA(myEnv, myClass,
    myMethod, (const jvalue *)&args);

  /* And check for exceptions */
  if ((tossed = (*myEnv)->ExceptionOccurred(myEnv)) != NULL) {
    fprintf(stderr, "%s: Exception detected:\n", argv[0]);
    (*myEnv)->ExceptionDescribe(myEnv);  /* writes on stderr */
    (*myEnv)->ExceptionClear(myEnv);  /* We're done with it. */
  }

  (*jvm)->DestroyJavaVM(jvm); /* no error checking as we're done anyhow */
  return 0;
}
```

1 *kwrite* is Unix-specific; it's a part of the K Desktop Environment (KDE).